UNITY 6000.0.62F1 Â· COMPLETE OFFLINE DOCUMENTATION

Blade Merge Developer Guide

Install. Edit. Reskin. Build your release.

szekipapa77@gmail.com



CHAPTER 01

Start here: Blade Merge

Blade Merge is a standalone editable Unity game template. Drag equal-tier swords together on the forge bench to merge them. Buy additional blades when coins permit, start the fight and tap to throw one blade at a time. Matching a blade already in a spinning wheel merges it; mismatched collisions knock the thrown blade away. Crystal golems require carving through their heart crystals before they reach the rampart.

Includes 25 curated levels, source-backed generation, a game-specific visual editor and validation, 4 guided helpers, local saving and portrait/landscape scenes. Keep the required private game Kit and publisher-owned Szekipapa77Ads layer. Provider SDK binaries are excluded. There is no bundled HLK, purchases, multiplayer server or remote LiveOps backend. Import only one game Kit into a project because the Kit classes share names.

CHAPTER 02

Requirements, installation and first play

Use Unity **6000.0.62f1**, URP **17.0.4** and Unity UI **2.0.0**. Install matching Android SDK/NDK/OpenJDK and WebGL modules in Unity Hub. Retain the supplied manifest and ProjectSettings.

1. Extract the standalone project to a short writable local path, then open it in the matching editor. Wait for package import and compilation.
2. For native unitypackage import, create a matching URP project and use Assets > Import Package > Custom Package. Import every game, Kit, Szekipapa77Ads and WebGL-template entry.
3. Run Szekipapa77 > Blade Merge > Setup. It configures rendering and player settings; review its changes when integrating with existing work.
4. Active Input Handling must support the classic Input Manager: choose Old or Both. New Input System only is insufficient.
5. Open `Assets/Games/BladeMerge/Scenes/BladeMerge_Mobile.unity` and press Play. Follow the level card and first-level instructions.
6. Stop Play mode before editing level assets. Keep all .meta GUIDs. The game uses classic Unity UI Text; do not assume an extra TextMesh Pro import is required.

CHAPTER 03

Project folders and exact source map

All game-relative paths below start at `Assets/Games/BladeMerge`. Runtime UI is built in code; editing an unrelated static Canvas does not reskin it. Scripts contains game rules and views; Editor contains authoring tools; Kit supplies UI/audio/save/platform support; Resources/blademerge contains level JSON, art, music and sound; Settings owns URP assets; Scenes contains both entry scenes.

- `Scripts/BladeMergeGame.cs`
- `Scripts/BladeMergeGame.UI.cs`
- `Scripts/BladeMergeGame.View.cs`
- `Scripts/BMBot.cs`
- `Scripts/BMData.cs`
- `Scripts/BMLevels.cs`
- `Scripts/BMMeshes.cs`
- `Scripts/BMSim.cs`

Main flow: BladeMergeGame.cs. UI and popup consumers: BladeMergeGame.UI.cs. Model/generator/loading: BMData.cs, BMLevels.cs, BMSim.cs. Authoring: Editor/BladeMergeLevelEditor.cs. Shared ad settings live outside the game at Assets/Szekipapa77Ads/Resources/AdsSettings.asset.

CHAPTER 04

Gameplay, controls and progression

Drag equal-tier swords together on the forge bench to merge them. Buy additional blades when coins permit, start the fight and tap to throw one blade at a time. Matching a blade already in a spinning wheel merges it; mismatched collisions knock the thrown blade away. Crystal golems require carving through their heart crystals before they reach the rampart.

The initial bank has 25 curated levels. Read the supplied loader/generator before changing continuation rules. Play the guided opening, ordinary progress, completion and retry paths; preserve saved indexes during a visual reskin. The Mobile scene is portrait and the WebGL-named scene is landscape, but the supplied Build menu selects portrait for both build targets. Select landscape explicitly in your Build Profile if desired. Test a near-startup scene replacement as well as a normal restart.

CHAPTER 05

Tutorials and every helper

Fresh local progress triggers the first-level guidance. Newly unlocked helpers have introduction and guided-use flows. Reset only a test save when reviewing tutorials. A free guided first use can differ from ordinary charge consumption. Exact names/unlocks/free charges: Scripts/BladeMergeGame.cs; effect/help strings: Scripts/BladeMergeGame.cs.

Helper	Unlock	Free charges	Action and effect
Whetstone	3	3	Your next 3 blades hit twice as hard.
Frost Bell	6	2	Freezes the wheel or golem in place for 5 seconds.
Auto-Forge	10	2	At the forge: spends your coins and merges your blades for you.
Meteor	15	2	A falling blade blasts 30% off a wheel, or a whole column off a golem.

Activate the tool and perform its required target action: arming a tool alone does not demonstrate its effect. A successful use requires the stated preconditions from this game: show both activation and its actual effect. Exhausted charges may offer a rewarded ad; verify success/cancel/failure and grant a reward once only.

CHAPTER 06

Visual level editor: worked editing session

Open **Szekipapa77 > Blade Merge > Level Editor**. Exact implementation: `Editor/BladeMergeLevelEditor.cs`.

Open the Blade Merge editor and select an early level. Use Dup or Duplicate before editing. Change its visible name and one rule or board element with the exact controls below. Save explicitly and run the current entry through the editor validation and play controls. The per-game source-control table includes the current brush, geometry, target, rule and AI fields; do not treat hierarchy or resource IDs as display text.

1. Back up Resources/blademerge/Levels/levels.json.
2. Duplicate a simple early level and edit one small element.
3. Use the game-specific validation controls and resolve every reported issue.
4. Save explicitly; stop and restart Play mode, then use Play this level or navigate to the edited index.
5. Check portrait and landscape and rebuild demo outputs after accepting data changes.

The exact current authoring controls, including tooltips and generator switches, follow. These are game-specific source references, not a generic editor feature list.

Source	Exact current code
`BladeMergeLevelEditor.cs:49`	`[MenuItem("Szekipapa77/Blade Merge/Level Editor")]`
`BladeMergeLevelEditor.cs:52`	`[MenuItem("Szekipapa77/Blade Merge/Validate All Levels")]`
`BladeMergeLevelEditor.cs:119`	`if (GUILayout.Button(_dirty ? "Save *" : "Save", EditorStyles.toolbarButton, GUILayout.Width(70))) Save();`
`BladeMergeLevelEditor.cs:120`	`if (GUILayout.Button("Reload", EditorStyles.toolbarButton, GUILayout.Width(60)) && (!_dirty || EditorUtility.DisplayDialog("Reload", "Discard unsaved changes?", "Discard", "Cancel"))) Load();`
`BladeMergeLevelEditor.cs:123`	`if (GUILayout.Button("Validate", EditorStyles.toolbarButton, GUILayout.Width(70)) && Cur != null) ValidateOne(_sel);`
`BladeMergeLevelEditor.cs:124`	`if (GUILayout.Button("Validate All", EditorStyles.toolbarButton, GUILayout.Width(85))) ValidateAll();`

BladeMergeLevelEditor.cs:129	if (GUILayout.Button(new GUIContent("Quick test", "Optional: saves, opens the WebGL scene at this level and enters Play mode. Editing never needs it."), EditorStyles.toolbarButton, GUILayout.Width(80)) && Cur != null) Play(_sel + 1);
BladeMergeLevelEditor.cs:156	if (GUILayout.Button("Add")) { var l = BMBot.Generate(new BMBot.GenParams { Level = _file.levels.Length + 1, Difficulty = 0.3f, Seed = Environment.TickCount, Validate = false }); l.name = "New Level"; Insert(_sel + 1, l); }
BladeMergeLevelEditor.cs:157	if (GUILayout.Button("Dup") && Cur != null) { var c = Cur.Clone(); c.name += " copy"; Insert(_sel + 1, c); }
BladeMergeLevelEditor.cs:158	if (GUILayout.Button("Del") && Cur != null && EditorUtility.DisplayDialog("Delete", $"Delete level {_sel + 1} '{Cur.name}'?", "Delete", "Cancel"))
BladeMergeLevelEditor.cs:162	if (GUILayout.Button("Up") && _sel > 0) Move(-1);
BladeMergeLevelEditor.cs:163	if (GUILayout.Button("Down") && _sel < _file.levels.Length - 1) Move(1);
BladeMergeLevelEditor.cs:185	EditorGUILayout.LabelField($"Level {_sel + 1}", EditorStyles.boldLabel);
BladeMergeLevelEditor.cs:187	l.name = EditorGUILayout.TextField("Name", l.name);
BladeMergeLevelEditor.cs:188	l.theme = EditorGUILayout.Popup("Theme", l.theme, BM.ThemeName, GUILayout.Width(300));
BladeMergeLevelEditor.cs:191	l.difficulty = EditorGUILayout.Slider("Design difficulty", l.difficulty, 0, 1);
BladeMergeLevelEditor.cs:192	l.breather = EditorGUILayout.ToggleLeft("Breather", l.breather, GUILayout.Width(80));
BladeMergeLevelEditor.cs:193	l.tut = EditorGUILayout.Popup(l.tut, new[] { "no tutorial", "throw tutorial", "merge tutorial", "forge tutorial" }, GUILayout.Width(120));

BladeMergeLevelEditor.cs:197	`l.intro = Intros[EditorGUILayout.Popup("Introduces (popup)", ii, Intros.Select(s => s == "" ? "(none)" : s).ToArray(), GUILayout.Width(360))];`
BladeMergeLevelEditor.cs:198	`l.hint = EditorGUILayout.TextField("First-time tip", l.hint);`
BladeMergeLevelEditor.cs:207	`EditorGUILayout.HelpBox($"Bot: {v} ({v.Seeds} seeds){(v.Solvable ? "" : "\nNOT reliably beatable: add coins / starting blades, or soften the encounters.")}", v.Solvable ? MessageType.Info : MessageType.Warning);`
BladeMergeLevelEditor.cs:213	`EditorGUILayout.LabelField("Forge bench", EditorStyles.boldLabel);`
BladeMergeLevelEditor.cs:216	`l.coins = EditorGUILayout.IntField("Coins", l.coins);`
BladeMergeLevelEditor.cs:217	`l.cost = Mathf.Max(1, EditorGUILayout.IntField("Forge price", l.cost));`
BladeMergeLevelEditor.cs:218	`l.costStep = Mathf.Max(0, EditorGUILayout.IntField("Price step", l.costStep));`
BladeMergeLevelEditor.cs:219	`l.forgeTier = Mathf.Clamp(EditorGUILayout.IntField("Forged tier", l.forgeTier), 1, BM.MaxTier);`
BladeMergeLevelEditor.cs:222	`int cols = EditorGUILayout.IntSlider("Columns", l.cols, 2, 5);`
BladeMergeLevelEditor.cs:223	`int rows = EditorGUILayout.IntSlider("Rows", l.rows, 2, 5);`
BladeMergeLevelEditor.cs:236	`if (_benchBrush == BenchBrush.Blade) _benchTier = EditorGUILayout.IntSlider("tier", _benchTier, 1, BM.MaxTier, GUILayout.Width(260));`
BladeMergeLevelEditor.cs:273	`EditorGUILayout.LabelField($"Starting damage {f.TotalDamage()} (+ {l.coins} coins = about {l.coins / Mathf.Max(1, l.cost)} forged blades of tier {l.forgeTier}). Blades fly in reading order (top-left first). Right-click clears.", EditorStyles.wordWrappedMiniLabel);`

`BladeMergeLevelEditor.cs:281`	`EditorGUILayout.LabelField("Encounters (in order)", EditorStyles.boldLabel);`
`BladeMergeLevelEditor.cs:286`	`if (GUILayout.Button("+ Wheel", GUILayout.Width(70)) && l.stages.Length < 4) { var rng = new Hyperoza.HzRng(Environment.TickCount); var st = BMBot.GenWheel(rng, _sel + 1, l.difficulty); l.stages = l.stages.Append(st).ToArray(); _stageSel = l.stages.Length - 1; Mark(); }`
`BladeMergeLevelEditor.cs:287`	`if (GUILayout.Button("+ Golem", GUILayout.Width(70)) && l.stages.Length < 4) { var rng = new Hyperoza.HzRng(Environment.TickCount); var st = BMBot.GenGolem(rng, _sel + 1, l.difficulty, false); l.stages = l.stages.Append(st).ToArray(); _stageSel = l.stages.Length - 1; Mark(); }`
`BladeMergeLevelEditor.cs:288`	`if (GUILayout.Button("Remove", GUILayout.Width(70)) && l.stages.Length > 1) { var list = l.stages.ToList(); list.RemoveAt(_stageSel); l.stages = list.ToArray(); _stageSel = Mathf.Clamp(_stageSel, 0, l.stages.Length - 1); Mark(); }`
`BladeMergeLevelEditor.cs:289`	`if (GUILayout.Button("<", GUILayout.Width(24)) && _stageSel > 0) { (l.stages[_stageSel], l.stages[_stageSel - 1]) = (l.stages[_stageSel - 1], l.stages[_stageSel]); _stageSel--; Mark(); }`
`BladeMergeLevelEditor.cs:290`	`if (GUILayout.Button(">", GUILayout.Width(24)) && _stageSel < l.stages.Length - 1) { (l.stages[_stageSel], l.stages[_stageSel + 1]) = (l.stages[_stageSel + 1], l.stages[_stageSel]); _stageSel++; Mark(); }`
`BladeMergeLevelEditor.cs:293`	`if (s0 == null) { EditorGUILayout.HelpBox("A level needs at least one encounter.", MessageType.Warning); return; }`
`BladeMergeLevelEditor.cs:296`	`s0.name = EditorGUILayout.TextField("Encounter name", s0.name);`
`BladeMergeLevelEditor.cs:297`	`s0.reward = EditorGUILayout.IntField(new GUIContent("Coins after win", "Given at the forge break before the next encounter"), s0.reward, GUILayout.Width(260));`

`BladeMergeLevelEditor.cs:308`	`s.hp = Mathf.Max(1, EditorGUILayout.IntField("HP", s.hp));`
`BladeMergeLevelEditor.cs:309`	`s.spin = EditorGUILayout.Slider("Spin (deg/s, sign = dir)", s.spin, -260, 260);`
`BladeMergeLevelEditor.cs:312`	`s.spinMode = EditorGUILayout.Popup("Mode", s.spinMode, new[] { "Steady", "Surge", "Reverse", "Pendulum" });`
`BladeMergeLevelEditor.cs:313`	`s.spinPeriod = EditorGUILayout.Slider("Period (s)", s.spinPeriod, 1, 10);`
`BladeMergeLevelEditor.cs:314`	`if (s.spinMode == (int)Spin.Pendulum) s.spinAmp = EditorGUILayout.Slider("Swing (deg)", s.spinAmp, 60, 300);`
`BladeMergeLevelEditor.cs:321`	`if (_rimBrush == RimBrush.FoeBlade) _rimTier = EditorGUILayout.IntSlider("tier", _rimTier, 1, BM.MaxTier, GUILayout.Width(240));`
`BladeMergeLevelEditor.cs:322`	`if (_rimBrush == RimBrush.CritArc) _arcWidth = EditorGUILayout.Slider("width", _arcWidth, 10, 90, GUILayout.Width(240));`
`BladeMergeLevelEditor.cs:324`	`_scrub = EditorGUILayout.Slider("Preview time (s)", _scrub, 0, 20);`
`BladeMergeLevelEditor.cs:391`	`foreach (var it in items) if (!BM.RimReachable(s, it.Angle)) { EditorGUILayout.HelpBox($"Rim item at {it.Angle:0} deg never faces the thrower with this spin (pendulum swing too small).", it.Kind == Rim.Lock ? MessageType.Error : MessageType.Warning); break; }`
`BladeMergeLevelEditor.cs:392`	`EditorGUILayout.LabelField("Left-click the rim to place the brush, right-click to remove. Scrub the preview time to see where the rim will be when you throw.", EditorStyles.wordWrappedMiniLabel);`
`BladeMergeLevelEditor.cs:400`	`s.bhp = Mathf.Max(1, EditorGUILayout.IntField("Stone hp", s.bhp));`
`BladeMergeLevelEditor.cs:401`	`s.heartHp = Mathf.Max(1, EditorGUILayout.IntField("Heart hp", s.heartHp));`

BladeMergeLevelEditor.cs:402	s.armor = Mathf.Max(1, EditorGUILayout.IntField("Iron armour (damage needed)", s.armor));
BladeMergeLevelEditor.cs:405	s.speed = EditorGUILayout.Slider("March speed", s.speed, 0.1f, 1f);
BladeMergeLevelEditor.cs:406	s.dist = EditorGUILayout.Slider("Start distance", s.dist, 6, 16);
BladeMergeLevelEditor.cs:409	s.sway = EditorGUILayout.Popup("Sway", s.sway, new[] { "None", "Sine", "Step", "Zigzag" });
BladeMergeLevelEditor.cs:410	s.swayAmp = EditorGUILayout.Slider("Amplitude", s.swayAmp, 0, 3);
BladeMergeLevelEditor.cs:411	s.swayPeriod = EditorGUILayout.Slider("Period", s.swayPeriod, 1.5f, 10);
BladeMergeLevelEditor.cs:413	int w = EditorGUILayout.IntSlider("Columns", s.w, 1, 9), h = EditorGUILayout.IntSlider("Rows (depth)", s.h, 1, 8);
BladeMergeLevelEditor.cs:453	if (g.HeartsTotal == 0) EditorGUILayout.HelpBox("No heart crystal: the golem can't be beaten. Paint at least one heart.", MessageType.Error);
BladeMergeLevelEditor.cs:454	for (int c = 0; c < s.w; c++) { bool heart = false; for (int r = 0; r < s.h; r++) if (cells[r * s.w + c] == Blk.Heart) heart = true; if (heart && !reach[c]) { EditorGUILayout.HelpBox($"A heart in column {c + 1} never passes your lane. Add sway or move it.", MessageType.Warning); break; } }
BladeMergeLevelEditor.cs:461	_showGen = EditorGUILayout.Foldout(_showGen, "Auto Generate (seeded, bot-validated, same generator as the runtime)", true);
BladeMergeLevelEditor.cs:464	_genCount = EditorGUILayout.IntField("Count", _genCount); _genStart = EditorGUILayout.IntField("Start at level #", _genStart);
BladeMergeLevelEditor.cs:465	_genSeed = EditorGUILayout.IntField("Seed", _genSeed);

`BladeMergeLevelEditor.cs:468`	`_genFrom = EditorGUILayout.Slider("Start difficulty", _genFrom, 0, 1); _genTo = EditorGUILayout.Slider("End difficulty", _genTo, 0, 1);`
`BladeMergeLevelEditor.cs:471`	`_genBreather = EditorGUILayout.IntSlider("Breather every", _genBreather, 0, 8);`
`BladeMergeLevelEditor.cs:472`	`_genReplace = EditorGUILayout.ToggleLeft("Replace that range (else insert)", _genReplace);`
`BladeMergeLevelEditor.cs:483`	`if (GUILayout.Button($"Generate {_genCount} level(s)", GUILayout.Height(26))) Generate();`

CHAPTER 07

Generator, insertion and replacement

Use the actual generation panel and its fields listed below. Set a reproducible seed and a small count, then review the start/index and replace option before clicking Generate. The exact generation implementation is reproduced separately with line references so append, insert and replacement behavior can be distinguished. Save and reopen the data file to verify the intended entry count; runtime generated continuation is not a project-asset save.

Source	Exact current code
`BladeMergeLevelEditor.cs:47`	`bool _genReplace, _showGen = true;`
`BladeMergeLevelEditor.cs:472`	`_genReplace = EditorGUILayout.ToggleLeft("Replace that range (else insert)", _genReplace);`
`BladeMergeLevelEditor.cs:501`	`if (_genReplace) { int rm = Mathf.Min(_genCount, list.Count - at); if (rm > 0) list.RemoveRange(at, rm); }`
`BladeMergeLevelEditor.cs:502`	`list.InsertRange(at, made);`

Worked extension: back up the bank, generate a small set at moderate difficulty using the exact per-game controls, validate them, then Save. Reopen the JSON and confirm the count changed. Do not assume every editor has the same insertion behavior: the actual implementation below distinguishes insert, append and replace. Runtime continuation does not automatically append new records to project JSON. Validation and bot scores help find problems but do not certify a pleasant human difficulty curve.

CHAPTER 08

Actual level JSON and identifiers

Canonical asset: `Assets/Games/BladeMerge/Resources/blademerge/Levels/levels.json`. Array order controls visible numbering. Preserve numeric/boolean types, root version fields where present and resource/save identifiers. The actual first record is shown below; use the editor for structured coordinates and block/word data.


```
{
 "name": "First Throw",
 "theme": 0,
 "difficulty": 0.05,
 "breather": false,
 "intro": "",
 "hint": "",
 "tut": 1,
 "cols": 3,
 "rows": 3,
 "forge": "1,1,1,1,1,1,,,",
 "coins": 0,
 "cost": 5,
 "costStep": 0,
 "forgeTier": 1,
 "stages": [
  {
   "Kind": 0,
   "name": "Practice Millstone",
   "kind": 0,
   "reward": 0,
   "hp": 4,
   "spin": 55,
   "spinMode": 0,
   "spinPeriod": 5.1596823,
   "spinAmp": 229.74832,
   "rim": "",
   "crits": "",
   "w": 5,
   "h": 3,
   "cells": "",
   "bhp": 2,
   "armor": 8,
   "heartHp": 4,
   "speed": 0.35,
   "dist": 12,
   "sway": 0,
   "swayAmp": 1.2,
   "swayPeriod": 4
  }
 ]
}
```


Root keys: levels. Inspect BMData.cs, BMLevels.cs, BMSim.cs for field semantics and validation. Keep visible names/hints distinct from technical keys. The game-specific board, wave, card, opponent or question records are gameplay data; UI translation is separate.

CHAPTER 09

Reskin recipe: interface colours

Stop Play mode and open `Scripts/BladeMergeGame.UI.cs`. Find `Ink`, currently `#2A2350`. Change that hex literal to `#42C9B0`, preserve its symbol/type, compile and reopen the scene. Review HUD, menus, helper cards, settings/results and both orientations. Art sprites and procedural fallback shapes can have additional colours; edit them deliberately.

Actual consumers of this symbol are listed first, then all current palette/hex assignments. Editor-only palettes do not control the runtime. Preserve state/goal contrast and rebuild after accepting a change.

Source	Exact current code
`BladeMergeGame.UI.cs:126 / void BuildHud()`	`_tipText = HzUI.Label("T", _tipPanel, "", 40, Ink); _tipText.rectTransform.Stretch(18);`
`BladeMergeGame.UI.cs:372 / void RefreshQuiver()`	`var tl = HzUI.Label("T", chip.transform, tiers[i].ToString(), i == 0 ? 44 : 36, t == 2 || t == 9 || t == 10 ? Ink : Color.white); tl.rectTransform.Stretch();`
`BladeMergeGame.UI.cs:398 / void RefreshBenchLabels()`	`var l = HzUI.Label("N", bg.transform, t > 0 ? t.ToString() : (_forge.Rune[i] ? "x2" : ""), 38, t == 2 || t == 9 || t == 10 ? Ink : Color.white);`
`BladeMergeGame.UI.cs:443 / void UpdateLabels()`	`lab.color = s.Tier == 2 || s.Tier == 9 || s.Tier == 10 ? Ink : Color.white;`

Source	Exact current code
`BladeMergeGame.cs:32 / class/static declarations`	`static readonly string[] BoostColor = { "#FF8A3D", "#3FB8FF", "#B57BFF", "#FF4F6A" };`
`BladeMergeGame.cs:151 / IEnumerator Intro(string key)`	`ForgePopup(it.title, it.msg, HzUI.Hex("#7A5CFF"), Art(it.art), ("Got it", HzUI.Hex("#3FC795"), () => closed = true));`
`BladeMergeGame.cs:333 / IEnumerator StageWonFlow()`	`ForgePopup("Cleared!", msg, HzUI.Hex("#FF8A3D"), Art("coin"), ("Next: " + _def.stages[_stage].name, HzUI.Hex("#3FC795"), () => closed = true));`
`BladeMergeGame.cs:353 / IEnumerator WinFlow()`	`var p = ForgePopup("Victory!", "\n\nLevel " + lv + " forged and won!",HzUI.Hex("#FF8A3D"), Art("win_badge"),`

`BladeMergeGame.cs:354 / IEnumerator WinFlow()`	`("Next Level", HzUI.Hex("#3FC795"), () => HzPlatform.Interstitial(GameId, () => LoadLevel(Level))));`
`BladeMergeGame.cs:373 / IEnumerator LoseFlow()`	`ForgePopup(title, msg, HzUI.Hex("#7A5CFF"), Art("lose_badge"),`
`BladeMergeGame.cs:374 / IEnumerator LoseFlow()`	`(offer, HzUI.Hex("#3FC795"), () => HzPlatform.Rewarded(GameId, "revive", ok => { if (ok) Revive(); else LoadLevel(Level, true); })),`
`BladeMergeGame.cs:375 / IEnumerator LoseFlow()`	`("Retry", HzUI.Hex("#FF8A3D"), () => LoadLevel(Level, true)));`
`BladeMergeGame.cs:378 / IEnumerator LoseFlow()`	`ForgePopup(title, msg, HzUI.Hex("#7A5CFF"), Art("lose_badge"), ("Retry", HzUI.Hex("#FF8A3D"), () => LoadLevel(Level, true)));`
`BladeMergeGame.cs:410 / void OnBooster(int b)`	`("Get 1 (ad)", HzUI.Hex("#3FC795"), () => HzPlatform.Rewarded(GameId, "booster_" + b, ok => { if (ok) { SetCharges(b, Charges(b) + 1); Toast("+1 " + BoostName[b]); HzAudio.Play(Sfx.Coin, 0.7f); } })),`
`BladeMergeGame.cs:411 / void OnBooster(int b)`	`("No thanks", HzUI.Hex("#8C84B8"), null));`
`BladeMergeGame.UI.cs:12 / class/static declarations`	`static readonly Color Ink = HzUI.Hex("#2A2350");`
`BladeMergeGame.UI.cs:13 / class/static declarations`	`static readonly Color Panel = HzUI.Hex("#241E46");`
`BladeMergeGame.UI.cs:14 / class/static declarations`	`static readonly Color Ember = HzUI.Hex("#FF8A3D");`
`BladeMergeGame.UI.cs:69 / void BuildHud()`	`_levelSub = HzUI.Label("Name", lp.transform, _def.name, 30, HzUI.Hex("#FFD7B5"), TextAnchor.MiddleCenter, false);`
`BladeMergeGame.UI.cs:74 / void BuildHud()`	`_settingsBtn = HzIcons.IconButton(_hud, HzIcons.Settings, HzUI.Hex("#4A3E86"), () => { if (_phase != Phase.Ended) ShowSettingsPopup(); }, 108);`

`BladeMergeGame.UI.cs:82 / void BuildHud()`	`else { var ci = HzUI.Panel("Coin", cp.transform, HzUI.Hex("#FFC23A"), 30); ci.raycastTarget = false; ci.rectTransform.Place(new Vector2(0, 0.5f), new Vector2(60, 60), new Vector2(48, 0)); }`
`BladeMergeGame.UI.cs:83 / void BuildHud()`	`_coinText = HzUI.Label("N", cp.transform, "0", 50, HzUI.Hex("#FFE2A0"));`
`BladeMergeGame.UI.cs:95 / void BuildHud()`	`var chip = HzUI.Panel("Stage" + i, _stageStrip, HzUI.Hex("#3A3270"), 36);`
`BladeMergeGame.UI.cs:102 / void BuildHud()`	`var link = HzUI.Panel("Link", _stageStrip, HzUI.Hex("#3A3270"), 4); link.raycastTarget = false;`
`BladeMergeGame.UI.cs:111 / void BuildHud()`	`_encName = HzUI.Label("Name", _encPanel, "", 36, HzUI.Hex("#FFD7B5"), TextAnchor.MiddleLeft);`
`BladeMergeGame.UI.cs:117 / void BuildHud()`	`_encFill = HzUI.Panel("Fill", barBg.transform, HzUI.Hex("#FF5A6A"), 15); _encFill.raycastTarget = false;`
`BladeMergeGame.UI.cs:154 / void BuildHud()`	`_forgeBtn = HzUI.Button("Forge", _forgeBar, "", HzUI.Hex("#B5612E"), Color.white, DoForge, 50, 40);`
`BladeMergeGame.UI.cs:159 / void BuildHud()`	`_forgeText = HzUI.Label("P", _forgeBtn.transform, "", 36, HzUI.Hex("#FFE2A0")); _forgeText.rectTransform.Place(new Vector2(0.5f, 0), new Vector2(300, 48), new Vector2(30, 14), new Vector2(0.5f, 0));`
`BladeMergeGame.UI.cs:160 / void BuildHud()`	`_fightBtn = HzUI.Button("Fight", _forgeBar, "FIGHT!", HzUI.Hex("#3FC795"), Color.white, StartFight, 66, 40);`
`BladeMergeGame.UI.cs:220 / void BuildHud()`	`var bd = HzUI.Panel("Badge", v.Root, HzUI.Hex("#FF4D6D"), 30); bd.raycastTarget = false;`
`BladeMergeGame.UI.cs:239 / void RefreshBoosters()`	`v.Inner.color = un ? (usable ? HzUI.Hex(BoostColor[b]) : Color.Lerp(HzUI.Hex(BoostColor[b]), HzUI.Hex("#4A4470"), 0.55f)) : HzUI.Hex("#4A4470");`

`BladeMergeGame.UI.cs:245 / void RefreshBoosters()`	`v.Badge.GetComponent<Image>().color = c > 0 ? HzUI.Hex("#FF4D6D") : HzUI.Hex("#3FC795");`
`BladeMergeGame.UI.cs:247 / void RefreshBoosters()`	`v.Caption.color = un ? Color.white : HzUI.Hex("#B8B0E0");`
`BladeMergeGame.UI.cs:248 / void RefreshBoosters()`	`v.Ring.color = _guide == b ? HzUI.Hex("#FFE066") : new Color(1, 1, 1, 0);`
`BladeMergeGame.UI.cs:263 / void RefreshPhaseUi()`	`_forgeBtn.GetComponent<Image>().color = can ? HzUI.Hex("#B5612E") : HzUI.Hex("#6A5A6E");`
`BladeMergeGame.UI.cs:272 / void RefreshPhaseUi()`	`img.color = i < _stage ? HzUI.Hex("#3FC795") : i == _stage ? Ember : HzUI.Hex("#3A3270");`
`BladeMergeGame.UI.cs:307 / void RefreshHpLabel()`	`_encFill.color = w.LocksLeft > 0 ? HzUI.Hex("#FFC24A") : HzUI.Hex("#FF5A6A");`
`BladeMergeGame.UI.cs:318 / void RefreshHpLabel()`	`_encFill.color = g.Danger > 0.75f ? HzUI.Hex("#FF4F4F") : HzUI.Hex("#3FE0FF");`
`BladeMergeGame.UI.cs:329 / void RefreshHpLabel()`	`_encFill.color = st.Kind == Kind.Wheel ? HzUI.Hex("#FF5A6A") : HzUI.Hex("#3FE0FF");`
`BladeMergeGame.UI.cs:377 / void RefreshQuiver()`	`var w = HzUI.Label("Whet", _quiverBar, "x2 x" + _enc.WhetLeft, 34, HzUI.Hex("#FFC24A"));`
`BladeMergeGame.UI.cs:400 / void RefreshBenchLabels()`	`if (t == 0 && _forge.Rune[i]) l.color = HzUI.Hex("#7FE8FF");`
`BladeMergeGame.UI.cs:444 / void UpdateLabels()`	`lab.transform.parent.GetComponent<Image>().color = s.Foe ? HzUI.Hex("#7A2A2A") : HzUI.Hex(BM.TierHex[Mathf.Clamp(s.Tier, 1, BM.MaxTier)]);`
`BladeMergeGame.UI.cs:493 / void LevelBanner()`	`var n = HzUI.Label("N", bg.transform, sub, 42, HzUI.Hex("#FFD7B5")); n.rectTransform.Place(new Vector2(0.5f, 0), new Vector2(800, 70), new Vector2(0, 18), new Vector2(0.5f, 0));`
`BladeMergeGame.UI.cs:590 / void CompleteGuide(int b)`	`Toast("Nice! " + BoostName[b] + " is yours", HzUI.Hex("#FFD7B5"), 1.3f, 0.6f);`
`BladeMergeGame.UI.cs:679 / HzPopup ForgePopup(string title, string message, Color accent, Sprite art, params (string label, Color color, Action action)[] buttons)`	`var card = HzUI.Panel("Card", dim.transform, HzUI.Hex("#2E2660"), 56);`

BladeMergeGame.UI.cs:682 / HzPopup ForgePopup(string title, string message, Color accent, Sprite art, params (string label, Color color, Action action)[] buttons)	var inner = HzUI.Panel("Inner", card.transform, HzUI.Hex("#3A3178"), 44); inner.raycastTarget = false;
BladeMergeGame.UI.cs:715 / HzPopup ForgePopup(string title, string message, Color accent, Sprite art, params (string label, Color color, Action action)[] buttons)	p.Message = HzUI.Label("Msg", card.transform, message, 42, HzUI.Hex("#F2ECFF"), TextAnchor.MiddleCenter, false);
BladeMergeGame.UI.cs:745 / void StarRow(HzPopup p, int stars)	var s = HzIcons.Icon(p.Card, HzIcons.Star, 110, i < stars ? HzUI.Hex("#FFC23A") : new Color(1, 1, 1, 0.2f));
BladeMergeGame.UI.cs:758 / void UnlockPopup(int b, Action done)	("Great!", HzUI.Hex("#3FC795"), done));
BladeMergeGame.UI.cs:764 / void ShowSettingsPopup()	var p = ForgePopup("Settings", "", HzUI.Hex("#7A5CFF"), null,
BladeMergeGame.UI.cs:765 / void ShowSettingsPopup()	(SoundLabel(), HzUI.Hex("#5B9DF0"), null),
BladeMergeGame.UI.cs:766 / void ShowSettingsPopup()	(MusicLabel(), HzUI.Hex("#A66BFF"), null),
BladeMergeGame.UI.cs:767 / void ShowSettingsPopup()	("How to play", HzUI.Hex("#3FB8FF"), ShowHowToPlay),
BladeMergeGame.UI.cs:768 / void ShowSettingsPopup()	("Restart Level", HzUI.Hex("#FF8A3D"), () => LoadLevel(Level, true)),
BladeMergeGame.UI.cs:769 / void ShowSettingsPopup()	("Close", HzUI.Hex("#3FC795"), null));
BladeMergeGame.UI.cs:788 / void ShowHowToPlay()	ForgePopup("How to play", HowToPlayText, HzUI.Hex("#3FB8FF"), null, ("Got it", HzUI.Hex("#3FC795"), null));
BladeMergeGame.View.cs:21 / class/static declarations	new Theme { SkyTop = "#1C1838", SkyBottom = "#B5527A", GroundA = "#6E5262", GroundB = "#5E4556", Cliff = "#33223A", WheelSide = "#8C5A3A", WheelTop = "#C88A52", Rune = "#FF9A3D", Stone = "#7C68B4", Stone2 = "#9A86D2", Spire = "#3A2A4E", Glow = "#FF7A2F", Sun = "#FFE2C4", AmbSky = "#8E7AC8", AmbEq = "#C88A9A", AmbGround = "#3A2A48" },

BladeMergeGame.View.cs:22 / class/static declarations	new Theme { SkyTop = "#14223E", SkyBottom = "#7FB2E0", GroundA = "#A7BED2", GroundB = "#93ADC4", Cliff = "#465E7C", WheelSide = "#5C7EA6", WheelTop = "#CDE4F4", Rune = "#7FE8FF", Stone = "#5866A8", Stone2 = "#7684C4", Spire = "#3C5676", Glow = "#7FE8FF", Sun = "#EAF4FF", AmbSky = "#9EC2E8", AmbEq = "#B8CCE0", AmbGround = "#3C5070" },
BladeMergeGame.View.cs:23 / class/static declarations	new Theme { SkyTop = "#0E2A26", SkyBottom = "#7CC79A", GroundA = "#7A9474", GroundB = "#6A8566", Cliff = "#2C4636", WheelSide = "#6E5C3E", WheelTop = "#B6A26C", Rune = "#5CFFB0", Stone = "#B0874E", Stone2 = "#C9A06A", Spire = "#2A4A3A", Glow = "#5CFFB0", Sun = "#F4FFE6", AmbSky = "#8CC8A6", AmbEq = "#A8C49A", AmbGround = "#2C4434" },
BladeMergeGame.View.cs:24 / class/static declarations	new Theme { SkyTop = "#120C26", SkyBottom = "#6A44A0", GroundA = "#544874", GroundB = "#483D66", Cliff = "#221A38", WheelSide = "#5E407E", WheelTop = "#A07ED8", Rune = "#D07BFF", Stone = "#4FA3A8", Stone2 = "#6CC2C4", Spire = "#2C1F48", Glow = "#E07BFF", Sun = "#F0E2FF", AmbSky = "#8A70C8", AmbEq = "#9C7CC0", AmbGround = "#261C40" },
BladeMergeGame.View.cs:25 / class/static declarations	new Theme { SkyTop = "#38202E", SkyBottom = "#FFB45A", GroundA = "#D8AA6E", GroundB = "#C89A5E", Cliff = "#7E5232", WheelSide = "#8C4C2C", WheelTop = "#E0B272", Rune = "#FFD24A", Stone = "#7A4E8E", Stone2 = "#9468A8", Spire = "#5E3A26", Glow = "#FFB03A", Sun = "#FFF0D8", AmbSky = "#E8B48A", AmbEq = "#E0A878", AmbGround = "#5A3A28" },
BladeMergeGame.View.cs:92 / void SetupLook()	_cam.backgroundColor = HzUI.Hex("#1C1838");
BladeMergeGame.View.cs:109 / void SetupLook()	fill.transform.rotation = Quaternion.Euler(26, 150, 0); fill.color = HzUI.Hex("#9FB4FF"); fill.intensity = 0.32f;
BladeMergeGame.View.cs:112 / void SetupLook()	rim.transform.rotation = Quaternion.Euler(14, 180, 0); rim.color = HzUI.Hex("#FFB07A"); rim.intensity = 0.35f;
BladeMergeGame.View.cs:137 / void MakeMaterials()	_hiltMat = Lit("#3A2A2A", 0.35f);
BladeMergeGame.View.cs:138 / void MakeMaterials()	_hiltFoeMat = Lit("#5A2E2A", 0.2f);

`BladeMergeGame.View.cs:139 / void MakeMaterials()`	`_gemFoeMat = Unlit(HzUI.Hex("#FF3B3B") * 2.2f);`
`BladeMergeGame.View.cs:152 / void MakeMaterials()`	`_benchMat = Lit("#5A3A2E", 0.35f);`
`BladeMergeGame.View.cs:153 / void MakeMaterials()`	`_slotMat = Lit("#3B2F45", 0.55f);`
`BladeMergeGame.View.cs:154 / void MakeMaterials()`	`_slotRuneMat = Unlit(HzUI.Hex("#3FE0FF") * 1.6f);`
`BladeMergeGame.View.cs:155 / void MakeMaterials()`	`_slotLockedMat = Lit("#1E1828", 0.2f);`
`BladeMergeGame.View.cs:156 / void MakeMaterials()`	`_slotHiMat = Unlit(HzUI.Hex("#FFC24A") * 1.6f);`
`BladeMergeGame.View.cs:157 / void MakeMaterials()`	`_lockMat = Lit("#E0A93A", 0.85f, 0.85f);`
`BladeMergeGame.View.cs:158 / void MakeMaterials()`	`_coinOrbMat = Unlit(HzUI.Hex("#FFC23A") * 2.2f);`
`BladeMergeGame.View.cs:160 / void MakeMaterials()`	`_critMat = Unlit(HzUI.Hex("#FF5A2A") * 2.0f);`
`BladeMergeGame.View.cs:161 / void MakeMaterials()`	`_ironMat = Lit("#4A505E", 0.75f, 0.85f);`
`BladeMergeGame.View.cs:162 / void MakeMaterials()`	`_goldMat = Lit("#FFC23A", 0.9f, 0.9f);`
`BladeMergeGame.View.cs:163 / void MakeMaterials()`	`_heartMat = Unlit(HzUI.Hex("#3FE0FF") * 2.6f);`
`BladeMergeGame.View.cs:164 / void MakeMaterials()`	`_eyeMat = Unlit(HzUI.Hex("#FFB13A") * 1.7f);`
`BladeMergeGame.View.cs:166 / void MakeMaterials()`	`_sparkOrange = Unlit(HzUI.Hex("#FF8A2F") * 2.6f);`
`BladeMergeGame.View.cs:167 / void MakeMaterials()`	`_sparkCyan = Unlit(HzUI.Hex("#3FE0FF") * 2.6f);`
`BladeMergeGame.View.cs:168 / void MakeMaterials()`	`_sparkGold = Unlit(HzUI.Hex("#FFD24A") * 2.6f);`

`BladeMergeGame.View.cs:169 / void MakeMaterials()`	`_shardIron = Lit("#4A505E", 0.7f, 0.8f);`
`BladeMergeGame.View.cs:170 / void MakeMaterials()`	`_shardGold = Lit("#FFC23A", 0.9f, 0.9f);`
`BladeMergeGame.View.cs:173 / void MakeMaterials()`	`_rampartMat = Lit("#4E4058", 0.3f);`
`BladeMergeGame.View.cs:176 / void MakeMaterials()`	`_pedestalMat = Lit("#3E3448", 0.4f);`
`BladeMergeGame.View.cs:178 / void MakeMaterials()`	`_groundA = Lit("#777777", 0.25f); _groundB = Lit("#666666", 0.25f); _cliffMat = Lit("#444444", 0.15f); _spireMat = Lit("#444444", 0.3f);`
`BladeMergeGame.View.cs:179 / void MakeMaterials()`	`_glowMat = Unlit(Color.white); _wheelSide = Lit("#888888", 0.45f); _wheelTop = Lit("#999999", 0.5f); _runeMat = Unlit(Color.white);`
`BladeMergeGame.View.cs:180 / void MakeMaterials()`	`_stoneMat = Lit("#888888", 0.6f); _stone2Mat = Lit("#999999", 0.6f); _boulderMat = Lit("#777777", 0.4f);`
`BladeMergeGame.View.cs:181 / void MakeMaterials()`	`_shardStone = Lit("#888888", 0.6f); _shardWheel = Lit("#888888", 0.5f);`
`BladeMergeGame.View.cs:246 / void BuildBackground()`	`_bgImg = HzUI.Img("Grad", go.transform, HzUI.Gradient(HzUI.Hex("#1C1838"), HzUI.Hex("#B5527A")), Color.white);`
`BladeMergeGame.View.cs:252 / void BuildBackground()`	`var d = HzUI.Img("Dot", layer, HzUI.Circle(32), i % 3 == 0 ? HzUI.Hex("#FFB07A") : HzUI.Hex("#FFFFFF"));`
`BladeMergeGame.View.cs:782 / void PlayEvents(List<Ev> evs)`	`FloatText(hitPos + Vector3.up * 0.5f, (e.Crit ? "CRIT " : "") + "-" + e.Value, e.Crit ? HzUI.Hex("#FF7A2F") : HzUI.Hex("#FFFFFF"));`
`BladeMergeGame.View.cs:822 / void PlayEvents(List<Ev> evs)`	`if (e.Kind == EvKind.Shielded) { FloatText(hp + Vector3.up * 0.6f, "Shielded! Break the locks", HzUI.Hex("#FFC24A")); PulseLocks(); }`
`BladeMergeGame.View.cs:823 / void PlayEvents(List<Ev> evs)`	`else if (_stuckViews.TryGetValue(e.Index, out var sv)) { HzTween.Punch(sv.T, 0.25f, 0.2f); FloatText(hp + Vector3.up * 0.6f, "Bounced!", HzUI.Hex("#FF6A6A")); }`

`BladeMergeGame.View.cs:835 / void PlayEvents(List<Ev> evs)`	`FloatText(hp + Vector3.up * 0.6f, e.Value > 0 ? "Lock broken! " + e.Value + " left" : "Shield down!", HzUI.Hex("#FFC24A"));`
`BladeMergeGame.View.cs:845 / void PlayEvents(List<Ev> evs)`	`FloatText(pos + Vector3.up * 0.5f, e.Value == (int)Rim.Coin ? "+" + _enc.CoinValue + " coins" : "Tempered!", HzUI.Hex(e.Value == (int)Rim.Coin ? "#FFD24A" : "#FFFFFF"));`
`BladeMergeGame.View.cs:868 / void PlayEvents(List<Ev> evs)`	`if (e.Block == Blk.Gold) { HzAudio.Play(Sfx.Coin, 0.8f); SparkBurst(pos, _sparkGold, 14, 3f); FloatText(pos + Vector3.up * 0.5f, "+" + _enc.CoinValue + " coins", HzUI.Hex("#FFD24A")); }`
`BladeMergeGame.View.cs:877 / void PlayEvents(List<Ev> evs)`	`FloatText(pos + Vector3.up * 0.6f, "Too weak! Need " + ((GolemFight)_enc).S.armor, HzUI.Hex("#FFC24A"));`
`BladeMergeGame.View.cs:885 / void PlayEvents(List<Ev> evs)`	`FloatText(new Vector3(e.X, 1.2f, e.Z), e.Value > 0 ? "Heart broken! " + e.Value + " left" : "Last heart!", HzUI.Hex("#7FE8FF"));`
`BladeMergeGame.View.cs:900 / void PlayEvents(List<Ev> evs)`	`if (_enc is WheelFight) FloatText(pos + Vector3.up * 1f, "-" + e.Value, HzUI.Hex("#FF4F6A"));`
`BladeMergeGame.View.cs:1001 / IEnumerator EncounterDefeatedFx()`	`FloatText(new Vector3(0, 2.2f, _wheelRoot ? BM.WheelZ : (_enc is GolemFight g ? g.Z : 6)), "Defeated!", HzUI.Hex("#FFD24A"));`
`BladeMergeGame.View.cs:1005 / void WhetFx() { if (_nextView != null) SparkBurst(_nextView.T.position, _sparkOrange, 18, 2.5f); Toast("Next 3 blades: double damage!", HzUI.Hex("#FFC24A"), 1.2f, 0.8f); }`	`void WhetFx() { if (_nextView != null) SparkBurst(_nextView.T.position, _sparkOrange, 18, 2.5f); Toast("Next 3 blades: double damage!", HzUI.Hex("#FFC24A"), 1.2f, 0.8f); }`
`BMData.cs:113 / public static int Damage(int tier) => tier <= 0 ? 0 : 1 << Mathf.Clamp(tier - 1, 0, 20);`	`public static readonly string[] TierHex = { "#FFFFFF", "#D08B4E", "#B9C6D6", "#45C78A", "#E8455A", "#3E7BFF", "#A35CFF", "#FFC23A", "#FF7A2F", "#7FE8FF", "#FFFFFF" };`

CHAPTER 10

Reskin recipe: visible menu and HUD text

Open `Scripts/BladeMergeGame.UI.cs`, find the displayed literal `"Settings"`, and replace its displayed value with `"Options"`. Keep hierarchy names and technical IDs unchanged. In HzUI.Label/Button calls the first string may be an object name; change the displayed text argument. Read the method references below to distinguish construction from later refresh assignments.

Preserve interpolation braces, invariant number formatting, ON/OFF updates and line breaks. Update both initial and refresh paths, portrait and landscape. Exact names/unlocks/free charges: Scripts/BladeMergeGame.cs; effect/help strings: Scripts/BladeMergeGame.cs. Level names/hints remain separate JSON content. If a title appears in art, Title property or another screen, update those visible uses deliberately without renaming GameId, namespace, resources or save keys.

Hard-coded game strings do not automatically use shared localization. To translate the interface, route every display assignment through your translation table. Dictionaries, word banks and theme pools are independent gameplay assets and need separate validation. Test help, settings, results, tutorial and helper introductions after a fresh launch.

Source	Exact current code
`BladeMergeGame.cs:29 / class/static declarations`	`static readonly string[] BoostName = { "Whetstone", "Frost Bell", "Auto-Forge", "Meteor" };`
`BladeMergeGame.cs:151 / IEnumerator Intro(string key)`	`ForgePopup(it.title, it.msg, HzUI.Hex("#7A5CFF"), Art(it.art), ("Got it", HzUI.Hex("#3FC795"), () => closed = true));`
`BladeMergeGame.cs:171 / void StartFight()`	`if (_forge.BladeCount() == 0) { Toast("Forge a blade first!"); HzAudio.Play(Sfx.Wrong, 0.5f); return; }`
`BladeMergeGame.cs:172 / void StartFight()`	`if (_tut == 3 && _tutStep == 1) { HzAudio.Play(Sfx.Wrong, 0.5f); Toast("Forge a blade first!"); return; }`
`BladeMergeGame.cs:173 / void StartFight()`	`if (_tut == 2 && _tutStep == 1) { HzAudio.Play(Sfx.Wrong, 0.5f); Toast("Merge two blades first!"); return; }`
`BladeMergeGame.cs:256 / void HandleInput()`	`else if (s >= 0 && _forge.Locked[s]) { HzAudio.Play(Sfx.Wrong, 0.4f); Toast("This slot is locked"); }`
`BladeMergeGame.cs:280 / void DoDrop(int from, int to)`	`if (_forge.Tier[to] > HzSave.GetInt("blademerge_besttier", 1)) { HzSave.SetInt("blademerge_besttier", _forge.Tier[to]); NewTierToast(_forge.Tier[to]); }`

BladeMergeGame.cs:298 / void DoForge()	if (_forge.FirstEmpty() < 0) { HzAudio.Play(Sfx.Wrong, 0.5f); Toast("Bench full! Drag equal blades together to merge."); PunchBench(); return; }
BladeMergeGame.cs:299 / void DoForge()	if (_forge.Coins < _forge.Price) { HzAudio.Play(Sfx.Wrong, 0.5f); Toast("Not enough coins"); if (_coinPill) HzTween.Shake(_coinPill, 10, 0.3f); return; }
BladeMergeGame.cs:333 / IEnumerator StageWonFlow()	ForgePopup("Cleared!", msg, HzUI.Hex("#FF8A3D"), Art("coin"), ("Next: " + _def.stages[_stage].name, HzUI.Hex("#3FC795"), () => closed = true));
BladeMergeGame.cs:353 / IEnumerator WinFlow()	var p = ForgePopup("Victory!", "\n\nLevel " + lv + " forged and won!",HzUI.Hex("#FF8A3D"), Art("win_badge"),
BladeMergeGame.cs:373 / IEnumerator LoseFlow()	ForgePopup(title, msg, HzUI.Hex("#7A5CFF"), Art("lose_badge"),
BladeMergeGame.cs:378 / IEnumerator LoseFlow()	ForgePopup(title, msg, HzUI.Hex("#7A5CFF"), Art("lose_badge"), ("Retry", HzUI.Hex("#FF8A3D"), () => LoadLevel(Level, true)));
BladeMergeGame.cs:402 / void OnBooster(int b)	if (!Unlocked(b)) { Toast("Unlocks at level " + BoostUnlock[b]); HzAudio.Play(Sfx.Wrong, 0.5f); if (_boost[b] != null) HzTween.Shake(_boost[b].Root, 10, 0.3f); return; }
BladeMergeGame.cs:405 / void OnBooster(int b)	if (BoostInBattle(b) && !battle) { Toast("Use " + BoostName[b] + " during a fight"); HzAudio.Play(Sfx.Wrong, 0.4f); return; }
BladeMergeGame.cs:406 / void OnBooster(int b)	if (!BoostInBattle(b) && _phase != Phase.Forge) { Toast("Use " + BoostName[b] + " at the forge"); HzAudio.Play(Sfx.Wrong, 0.4f); return; }
BladeMergeGame.cs:409 / void OnBooster(int b)	ForgePopup("Need a " + BoostName[b] + "?", "Watch a short ad to get one more.", HzUI.Hex(BoostColor[b]), Art(BoostArt[b]),
BladeMergeGame.cs:410 / void OnBooster(int b)	("Get 1 (ad)", HzUI.Hex("#3FC795"), () => HzPlatform.Rewarded(GameId, "booster_" + b, ok => { if (ok) { SetCharges(b, Charges(b) + 1); Toast("+1 " + BoostName[b]); HzAudio.Play(Sfx.Coin, 0.7f); } })),

`BladeMergeGame.cs:422 / void OnBooster(int b)`	`if (_enc.Frozen) { Toast("Already frozen"); return; }`
`BladeMergeGame.cs:432 / void OnBooster(int b)`	`if (!changed) { Toast("Nothing to forge or merge"); HzAudio.Play(Sfx.Wrong, 0.4f); return; }`
`BladeMergeGame.cs:443 / void OnBooster(int b)`	`if (r < 0) { Toast("No target"); return; }`
`BladeMergeGame.cs:516 / public override string DebugCommand(string cmd)`	`case "settings": ShowSettingsPopup(); return "ok";`
`BladeMergeGame.UI.cs:66 / void BuildHud()`	`_levelText = HzUI.Label("Lvl", lp.transform, "Level " + Level, 54, Color.white);`
`BladeMergeGame.UI.cs:69 / void BuildHud()`	`_levelSub = HzUI.Label("Name", lp.transform, _def.name, 30, HzUI.Hex("#FFD7B5"), TextAnchor.MiddleCenter, false);`
`BladeMergeGame.UI.cs:74 / void BuildHud()`	`_settingsBtn = HzIcons.IconButton(_hud, HzIcons.Settings, HzUI.Hex("#4A3E86"), () => { if (_phase != Phase.Ended) ShowSettingsPopup(); }, 108);`
`BladeMergeGame.UI.cs:83 / void BuildHud()`	`_coinText = HzUI.Label("N", cp.transform, "0", 50, HzUI.Hex("#FFE2A0"));`
`BladeMergeGame.UI.cs:99 / void BuildHud()`	`else { var t = HzUI.Label("N", chip.transform, (i + 1).ToString(), 40, Color.white); t.rectTransform.Stretch(); }`
`BladeMergeGame.UI.cs:111 / void BuildHud()`	`_encName = HzUI.Label("Name", _encPanel, "", 36, HzUI.Hex("#FFD7B5"), TextAnchor.MiddleLeft);`
`BladeMergeGame.UI.cs:113 / void BuildHud()`	`_encValue = HzUI.Label("Val", _encPanel, "", 36, Color.white, TextAnchor.MiddleRight);`
`BladeMergeGame.UI.cs:126 / void BuildHud()`	`_tipText = HzUI.Label("T", _tipPanel, "", 40, Ink); _tipText.rectTransform.Stretch(18);`
`BladeMergeGame.UI.cs:154 / void BuildHud()`	`_forgeBtn = HzUI.Button("Forge", _forgeBar, "", HzUI.Hex("#B5612E"), Color.white, DoForge, 50, 40);`
`BladeMergeGame.UI.cs:157 / void BuildHud()`	`var fLbl = HzUI.Label("L", _forgeBtn.transform, "Forge", 52, Color.white); fLbl.rectTransform.Place(new Vector2(0.5f, 1), new Vector2(300, 64), new Vector2(30, -12), new Vector2(0.5f, 1));`

`BladeMergeGame.UI.cs:159 / void BuildHud()`	`_forgeText = HzUI.Label("P", _forgeBtn.transform, "", 36, HzUI.Hex("#FFE2A0")); _forgeText.rectTransform.Place(new Vector2(0.5f, 0), new Vector2(300, 48), new Vector2(30, 14), new Vector2(0.5f, 0));`
`BladeMergeGame.UI.cs:160 / void BuildHud()`	`_fightBtn = HzUI.Button("Fight", _forgeBar, "FIGHT!", HzUI.Hex("#3FC795"), Color.white, StartFight, 66, 40);`
`BladeMergeGame.UI.cs:174 / void BuildHud()`	`_hpLabel = HzUI.Label("HP", _labels, "", 64, Color.white);`
`BladeMergeGame.UI.cs:199 / void BuildHud()`	`v.Btn = HzUI.Button("Booster" + b, parent, null, Color.white, Color.white, () => OnBooster(id), 40, 75);`
`BladeMergeGame.UI.cs:223 / void BuildHud()`	`v.BadgeText = HzUI.Label("N", bd.transform, "", 34, Color.white); v.BadgeText.rectTransform.Stretch();`
`BladeMergeGame.UI.cs:225 / void BuildHud()`	`v.Caption = HzUI.Label("Cap", v.Root, "", 30, Color.white);`
`BladeMergeGame.UI.cs:244 / void RefreshBoosters()`	`v.BadgeText.text = c > 0 ? c.ToString() : "+";`
`BladeMergeGame.UI.cs:246 / void RefreshBoosters()`	`v.Caption.text = un ? BoostName[b] : "Level " + BoostUnlock[b];`
`BladeMergeGame.UI.cs:306 / void RefreshHpLabel()`	`_encValue.text = Mathf.Max(0, w.Hp) + " / " + w.MaxHp + (w.LocksLeft > 0 ? " (locked)" : "");`
`BladeMergeGame.UI.cs:317 / void RefreshHpLabel()`	`_encValue.text = "Hearts " + g.HeartsLeft + " / " + g.HeartsTotal;`
`BladeMergeGame.UI.cs:326 / void RefreshHpLabel()`	`_encName.text = "Next: " + st.name;`
`BladeMergeGame.UI.cs:327 / void RefreshHpLabel()`	`_encValue.text = st.Kind == Kind.Wheel ? "HP " + st.hp : "Hearts " + BM.HeartCount(st);`
`BladeMergeGame.UI.cs:359 / void RefreshQuiver()`	`var lab = HzUI.Label("Left", _quiverBar, n == 0 ? "No blades left" : col ? "Blades\n" + n : "Blades: " + n, 34, Color.white, TextAnchor.MiddleCenter);`

`BladeMergeGame.UI.cs:372 / void RefreshQuiver()`	`var tl = HzUI.Label("T", chip.transform, tiers[i].ToString(), i == 0 ? 44 : 36, t == 2 || t == 9 || t == 10 ? Ink : Color.white); tl.rectTransform.Stretch();`
`BladeMergeGame.UI.cs:377 / void RefreshQuiver()`	`var w = HzUI.Label("Whet", _quiverBar, "x2 x" + _enc.WhetLeft, 34, HzUI.Hex("#FFC24A"));`
`BladeMergeGame.UI.cs:398 / void RefreshBenchLabels()`	`var l = HzUI.Label("N", bg.transform, t > 0 ? t.ToString() : (_forge.Rune[i] ? "x2" : ""), 38, t == 2 || t == 9 || t == 10 ? Ink : Color.white);`
`BladeMergeGame.UI.cs:433 / void UpdateLabels()`	`var l = HzUI.Label("N", bg.transform, "", 30, Color.white); l.rectTransform.Stretch();`
`BladeMergeGame.UI.cs:467 / void FloatText(Vector3 world, string text, Color c)`	`var l = HzUI.Label("Float", _labels, text, 54, c);`
`BladeMergeGame.UI.cs:490 / void LevelBanner()`	`var t = HzUI.Label("L", bg.transform, "Level " + Level, 92, Color.white); t.rectTransform.Place(new Vector2(0.5f, 1), new Vector2(800, 110), new Vector2(0, -16), new Vector2(0.5f, 1));`
`BladeMergeGame.UI.cs:493 / void LevelBanner()`	`var n = HzUI.Label("N", bg.transform, sub, 42, HzUI.Hex("#FFD7B5")); n.rectTransform.Place(new Vector2(0.5f, 0), new Vector2(800, 70), new Vector2(0, 18), new Vector2(0.5f, 0));`
`BladeMergeGame.UI.cs:529 / void NewTierToast(int tier)`	`Toast("New blade: " + BM.TierName[Mathf.Clamp(tier, 1, BM.MaxTier)] + "!", HzUI.Hex(BM.TierHex[Mathf.Clamp(tier, 1, BM.MaxTier)]), 1.4f, 0.42f);`
`BladeMergeGame.UI.cs:590 / void CompleteGuide(int b)`	`Toast("Nice! " + BoostName[b] + " is yours", HzUI.Hex("#FFD7B5"), 1.3f, 0.6f);`
`BladeMergeGame.UI.cs:710 / HzPopup ForgePopup(string title, string message, Color accent, Sprite art, params (string label, Color color, Action action)[] buttons)`	`var t = HzUI.Label("Title", ribbon.transform, title, 70, Color.white); t.rectTransform.Stretch(6);`
`BladeMergeGame.UI.cs:715 / HzPopup ForgePopup(string title, string message, Color accent, Sprite art, params (string label, Color color, Action action)[] buttons)`	`p.Message = HzUI.Label("Msg", card.transform, message, 42, HzUI.Hex("#F2ECFF"), TextAnchor.MiddleCenter, false);`

`BladeMergeGame.UI.cs:757 / void UnlockPopup(int b, Action done)`	`ForgePopup("New: " + BoostName[b] + "!", BoostDesc[b] + "\nYou get " + BoostStart[b] + " free!", HzUI.Hex(BoostColor[b]), Art(BoostArt[b]),`
`BladeMergeGame.UI.cs:764 / void ShowSettingsPopup()`	`var p = ForgePopup("Settings", "", HzUI.Hex("#7A5CFF"), null,`
`BladeMergeGame.UI.cs:765 / void ShowSettingsPopup()`	`(SoundLabel(), HzUI.Hex("#5B9DF0"), null),`
`BladeMergeGame.UI.cs:766 / void ShowSettingsPopup()`	`(MusicLabel(), HzUI.Hex("#A66BFF"), null),`
`BladeMergeGame.UI.cs:788 / void ShowHowToPlay()`	`ForgePopup("How to play", HowToPlayText, HzUI.Hex("#3FB8FF"), null, ("Got it", HzUI.Hex("#3FC795"), null));`
`BladeMergeGame.UI.cs:800 / static string SoundLabel() => HzAudio.Muted ? "Sound: OFF" : "Sound: ON";`	`static string SoundLabel() => HzAudio.Muted ? "Sound: OFF" : "Sound: ON";`
`BladeMergeGame.UI.cs:801 / static string MusicLabel() => HzAudio.MusicMuted ? "Music: OFF" : "Music: ON";`	`static string MusicLabel() => HzAudio.MusicMuted ? "Music: OFF" : "Music: ON";`
`BladeMergeGame.View.cs:1005 / void WhetFx() { if (_nextView != null) SparkBurst(_nextView.T.position, _sparkOrange, 18, 2.5f); Toast("Next 3 blades: double damage!", HzUI.Hex("#FFC24A"), 1.2f, 0.8f); }`	`void WhetFx() { if (_nextView != null) SparkBurst(_nextView.T.position, _sparkOrange, 18, 2.5f); Toast("Next 3 blades: double damage!", HzUI.Hex("#FFC24A"), 1.2f, 0.8f); }`
`BMData.cs:115 / public static int Damage(int tier) => tier <= 0 ? 0 : 1 << Mathf.Clamp(tier - 1, 0, 20);`	`public static readonly string[] ThemeName = { "Ember Crag", "Frost Hollow", "Jade Ruins", "Amethyst Deep", "Sunforge Dunes" };`

CHAPTER 11

Artwork, fonts, audio and licences

Replace game images under Resources/blademerge through Unity while retaining resource keys, .meta GUIDs, Sprite/UI import settings, alpha, pivots and slicing. Procedural graphics in the art/view sources may need separate colour changes. Replacing one PNG does not change every fallback. Keep third-party font and other licence notices with the corresponding shipped assets.

Effects are under Resources/blademerge/Sfx. Preserve each actual load key and importer settings when replacing sounds. Supplied music: Resources/blademerge/Music/forge_loop.ogg. Preserve that load key and verify loop/compression and the independent Music toggle. Kit UI uses supplied fonts; preserve actual licence notices and verify new glyphs.

CHAPTER 12

Local saves, reset and migration

HzSave wraps PlayerPrefs and flushes local state on focus loss/quit. WebGL persistence uses browser storage tied to the hosting origin. Clearing site data or changing origin starts a new local save; this is not cloud synchronization. Inspect HzSave/PlayerPrefs calls in the actual game source before altering progression, stars, helper charges or tutorial flags.

Verify fresh launch, a completed arena, helper use, sound options, focus loss and restart. Use a dedicated test profile for resets. Keep technical save keys stable during colour/text reskins. If changing schema or keys, implement and test a migration rather than silently discarding existing usersâ€™ progress.

CHAPTER 13

Advertising integration and configuration

Asset: `Assets/Szekipapa77Ads/Resources/AdsSettings.asset`. Inspect backend, useGoogleTestIds and missingSdkFallback in the publisher-owned Ads configuration. Buyer projects include adapter/integration code and exclude external provider SDK binaries. WebGL and a missing-provider fallback simulate ads; these are not live commercial impressions.

For real Android ads install the supported provider SDK from its official source into your working project, enter your own app/placement IDs and use test ads first. Verify banner/interstitial/rewarded callbacks on supported hardware. For production use your own IDs and disable simulated missing-SDK fallback. Build tooling rejects unsafe production fallback settings. No IAP, remote backend or HLK is included in this publisher edition.

CHAPTER 14

Android build and release

Install Unity 6000.0.62f1 Android modules through Hub. Select Android, run Setup and review application ID `com.szekipapa77.blademerge`, version, scene orientation and signing. The build command writes `Builds/Android/blademerge.apk`. Confirm BuildResult.Succeeded and the actual output file; intermediate report totals are not the final APK size.

Use your own keystore/signing and distribution settings. A test-signed demo is not a signed store release. A provider-free build simulates ads; a provider test-demo does not prove production ad-account configuration. Test real hardware touch, safe areas, focus/audio, save/relaunch and reward callbacks. Keep passwords, keystores and private service credentials out of buyer packages.

CHAPTER 15

WebGL build and hosting

Install matching WebGL support. Run Setup and Build WebGL; output is `Builds/WebGL/blademerge/` . Serve index.html with all loader/framework/data/WASM/StreamingAssets files over HTTP(S), not file://. Match compression, Content-Encoding and MIME types to the host. Outer page sizing and actual scene orientation are different settings.

The publisherâ€™s playable demo is hosted on HyperCodeStore. Upload the actual runtime files rather than merely changing a URL. Test a fresh anonymous load, real pointer controls, audio after interaction, saved progress/reload and phone/desktop sizing. Rebuild and rehost when code/assets change. A successful HTTP page alone is not gameplay verification.

CHAPTER 16

Troubleshooting

Compile errors: restore matching packages and the full private Kit/Ads layer; only one game Kit belongs in a project. **Pink/invisible rendering:** run Setup and inspect Graphics and every Quality URP assignment. **No input:** classic input handling must be enabled; check canvas focus and blocking modals. **Missing resources:** restore exact paths, .meta GUIDs and importer settings.

Edits do not appear: stop Play, Save in the level editor, reload the canonical bank and check current saved level. **Text reverts:** update construction and refresh assignments from the exact table. **No live ads:** provider binaries are deliberately excluded; inspect your installed provider and fallback mode. **Scene reload exception:** retain owner-lifetime and generation guards on delayed callbacks. **WebGL fails:** inspect browser Console, compression headers and missing payloads.

CHAPTER 17

Support and final release checklist

Publisher: Szekipapa77. Support: szekipapa77@gmail.com. Include game title, exact Unity version, platform, first Console error and reproduction steps. Never send passwords/tokens.

1. Clean-import the standalone project and native package.
2. Play both scenes and replace a scene near startup.
3. Review tutorial, normal completion/retry and every helper activation plus effect.
4. Validate editor changes and test reskin text/colours in both layouts.
5. Rebuild affected Android/WebGL outputs.
6. Verify complete public APK bytes, actual browser demo, documentation and video links.
7. Keep licences and exclude all internal prompts, planning/capture/build/QA tooling from buyers.